

1. Introduction

1.1 Background

Digital credentials are used to prove facts about a person, such as their educational qualifications, identity, or professional status. In traditional systems, a central authority is responsible for issuing, storing, and verifying these credentials. For example, when an employer wants to confirm that a job applicant holds a university degree, the typical process involves contacting the university directly, accessing a centralized verification portal, or requesting a physical or scanned copy of the degree certificate.

While these approaches are widely used, they carry a significant privacy limitation: they often require sharing the full credential, even when only a small portion of it is relevant. A clear example is Tribhuvan University in Nepal, where graduates must submit their full academic transcripts when applying for jobs or further education. This transcript contains all semester marks, individual subject grades, pass or fail results, and personal details, even when an employer only needs to confirm the applicant's degree title and graduating institution. This practice of sharing more information than required is known as over-disclosure, and it is a well-recognised concern in digital identity management.

The importance of addressing this problem is recognised at a policy level. The European Union's Digital Identity Wallet initiative requires that citizens be able to share only the minimum necessary personal data when verifying their credentials across digital services (European Commission, n.d.). This principle of data minimisation is also a core requirement of the General Data Protection Regulation (GDPR). While these regulations apply across EU member states, the underlying principle that individuals should control what personal data they share is universally relevant to any digital credential system, including the university degree verification context explored in this project.

This project applies these principles to a concrete and practical scenario: a university graduate holding a digital degree credential and presenting only the information needed to satisfy a specific employer's verification request. The system is built as a backend prototype using three independent Django web services, each representing one of the three core roles in the W3C Verifiable Credentials model: issuer, holder, and verifier (Sporny et al., 2025).

1.2 Problem Statement

Existing credential verification systems are largely centralized and offer limited control to the credential holder. A clear example is the current degree verification process at Tribhuvan University (TU) in Nepal. When graduates apply for employment or further education, they are required to submit a scanned copy of their full academic transcript, which contains all semester marks, individual subject

grades, pass or fail results, and personal details. However, an employer typically only needs to confirm that the applicant holds a degree in a specific subject from a recognised institution. The current process forces graduates to disclose their entire academic record, including information that is completely irrelevant to the verification request.

This unnecessary sharing of personal data is known as over-disclosure and represents a fundamental privacy limitation of centralized credential systems. A graduate should be able to prove they hold a Computer Science degree without revealing their semester marks, individual subject grades, or any other personal details that are not relevant to the specific request.

This study addresses that problem by designing and implementing a distributed digital credential verification prototype. In this system, a university acts as the issuer, a graduate acts as the holder, and an employer acts as the verifier. The system uses RSA digital signatures and salted SHA-256 hashes to ensure that credentials are authentic and tamper-proof, while allowing the holder to selectively disclose only the attributes required for a given verification interaction. This gives the graduate direct control over their personal data, addressing the over-disclosure problem present in existing centralized systems.

1.3 Research Questions

This study is guided by the following research questions:

RQ1: How can a distributed issuer-holder-verifier architecture support selective disclosure while protecting undisclosed credential attributes using RSA digital signatures and salted SHA-256 hashes?

RQ2: To what extent does the proposed prototype give the credential holder control over their personal data compared to the existing centralized degree verification process at Tribhuvan University?

1.4 Objectives

1.4.1 General Objective

The general objective of this study is to design and implement a distributed digital credential verification prototype that addresses the over-disclosure problem present in centralized credential systems, such as the degree verification process at Tribhuvan University. The system integrates the Django framework with RSA digital signatures, salted SHA-256 hashes, and selective disclosure mechanisms to give credential holders direct control over what personal information they share during a verification interaction.

1.4.2 Specific Objectives

- To design and implement a role-based credential verification system consisting of three primary entities: issuer (university), holder (graduate), and verifier (employer).
- To apply RSA digital signatures to ensure that issued credentials are authentic and tamper-proof.
- To use salted SHA-256 hashes to protect undisclosed credential attributes during selective presentation.
- To allow the holder to disclose only the attributes required for a specific verification interaction, while keeping remaining credential attributes hidden from the verifier.
- To evaluate the extent to which the proposed system gives the credential holder control over their personal data compared to existing centralized systems.

1.5 Scope

This study focuses on the design and implementation of a **backend** digital credential verification prototype developed using the Django framework. The proposed system adopts a role-based architecture consisting of three primary entities: issuer, holder, and verifier, representing a university, a graduate, and an employer respectively. The use case is specifically focused on university degree credential verification, where a graduate needs to prove their qualification to an employer without disclosing unnecessary personal attributes such as semester marks, individual subject grades, and other irrelevant details.

The system is implemented as a backend prototype only. It does not include a frontend user interface. Interactions between the three services are performed through HTTP API endpoints, demonstrating the core data flows and cryptographic processes of the system. This backend focus allows the project to concentrate on the fundamental technical and privacy aspects of selective disclosure without the added complexity of interface design.

The system uses RSA digital signatures and salted SHA-256 hashes to support secure credential issuance, selective disclosure, and verification. The three services communicate over HTTP and run as independent processes to simulate a distributed environment.

The scope of this work is limited to a prototype level implementation designed for academic and experimental purposes. It does not include:

- A dedicated frontend framework (e.g. React, Vue) — interactions use minimal Django-served HTML pages
- Full scale production deployment
- Blockchain integration
- Credential revocation mechanisms
- Holder authentication
- Zero knowledge proofs

- Non transferability of credentials

Instead the system focuses on demonstrating that selective disclosure is achievable at the backend level using currently available technologies, giving the credential holder meaningful control over what personal data is shared in a verification interaction.

1.6 Significance

This study is significant because it addresses a real and practical privacy problem faced by graduates in centralized credential systems. At institutions such as Tribhuvan University in Nepal, graduates currently have no control over what personal information is disclosed when their academic credentials are verified. When applying for employment or further education, graduates must submit their full academic transcript containing all semester marks, individual subject grades, and personal details, even when the employer or institution only needs to confirm a specific qualification.

This prototype demonstrates that this problem can be addressed at the backend level using currently available technologies such as Django, RSA digital signatures, and salted SHA-256 hashes, without requiring complex cryptographic constructs such as zero knowledge proofs.

The significance of this work can be understood from three perspectives.

From the perspective of graduates and students, the system shows how academic credentials could be issued and verified in a privacy respecting way. A graduate can prove they hold a specific degree without revealing their full academic transcript, giving them genuine control over their personal data.

From the perspective of employers and verifiers, the system demonstrates how credential authenticity can be confirmed through cryptographic signatures without needing access to an applicant's complete academic record. This reduces unnecessary personal data exposure during hiring and admission processes.

From the perspective of researchers and developers, this work provides a practical backend implementation of distributed credential verification principles. It also clearly identifies the limitations of the salt and hash approach, particularly regarding unlinkability, providing a foundation for future research into more advanced privacy preserving identity systems such as zero knowledge proofs and blockchain based identity frameworks.

2. Literature Review

2.1 Digital Credentials

A credential is a statement made by one party about another. A physical university degree certificate is a credential issued by the institution, asserting that a named individual has successfully completed a programme of study. Digital credentials are the electronic equivalent of this, structured in a way that can be transmitted, stored, and verified programmatically without requiring physical documents.

The World Wide Web Consortium (W3C) published a specification called Verifiable Credentials in 2019, which defines how digital credentials should work in a secure and privacy respecting manner (W3C, 2019). This specification defines five roles in a complete digital identity system. The issuer creates and signs the credential. The holder stores the credential and decides what to present. The verifier checks the credential when it is presented. The subject is the person the credential is about, who in most cases is also the holder. The verifiable data registry is a trusted system that stores issuer identifiers and public keys.

This prototype implements the three core roles of issuer, holder, and verifier. The subject and holder are represented by the same entity, the graduate, and the issuer's public key endpoint serves as a simplified verifiable data registry. The W3C specification confirms that these three roles are the minimum required to demonstrate a complete credential verification flow (Sporny et al., 2025).

The key advantage of the W3C Verifiable Credentials model is that the verifier does not need to contact the issuer at the time of verification. Instead, the issuer's digital signature on the credential is sufficient for the verifier to confirm its authenticity, provided the verifier has access to the issuer's public key. This enables decentralized verification without involving the issuer in every transaction.

2.2 Issuer-Holder-Verifier Model

The issuer-holder-verifier model is the foundation of modern digital credential systems. It separates the roles of credential creation, credential storage, and credential verification into three independent entities. This separation is central to giving the holder control over their personal data, as the issuer is not involved in any verification transaction after the initial issuance.

In the context of this project, the university acts as the issuer. When a student graduates, the university creates a digital degree credential containing the relevant claims, signs it with its RSA private key, and sends it directly to the graduate's digital wallet. From that point forward, the graduate holds the credential and decides when and what to share with an employer. The employer, acting as the verifier, checks the credential using the university's public key without needing to contact the university at all.

This model directly addresses the over-disclosure problem described in the problem statement. In the current TU verification process, the university is involved every time an employer requests verification, and the graduate has no control over what is shared. In the issuer-holder-verifier model, the graduate holds the credential and decides what to disclose in each transaction.

The EU Digital Identity Wallet initiative adopts this same model, requiring that EU citizens be able to store and selectively present their credentials from a personal digital wallet without depending on a central authority for each verification (European Commission, n.d.). This demonstrates that the issuer-holder-verifier model is not merely theoretical but is actively being adopted at a large scale in real world digital identity systems.

2.3 Selective Disclosure

Selective disclosure refers to the ability of a credential holder to reveal only specific attributes from a credential rather than presenting the entire document. This is one of the most important privacy properties in modern identity systems as it directly supports the principle of data minimisation, which states that only the data strictly necessary for a given purpose should be shared (Flamini et al., 2024).

Flamini et al. (2024) identify three main design patterns for implementing selective disclosure. The first is atomic credentials, where each claim is issued as a separate credential. While simple, this approach lacks a cryptographic link between claims and is generally considered unsuitable for practical use. The second is hiding commitment schemes, where all claims are included in a single credential but each claim is individually hashed with a random salt. Only the hash of an undisclosed claim is shared with the verifier, while the disclosed claim's value and salt are provided for hash recomputation. The third is SD signatures, which use advanced cryptographic methods such as zero knowledge proofs to allow claims to be proved without revealing the underlying values.

This project implements the hiding commitment approach, specifically a commitment list mechanism. Each claim in the degree credential is hashed with a unique random salt, the hashes are sorted deterministically, and the resulting string is signed by the issuer using RSA. This method is straightforward to implement using standard cryptographic libraries and is sufficient to demonstrate the core principles of selective disclosure without requiring advanced cryptographic constructs.

A practical example in the context of this project: when a TU graduate applies for a job, they can choose to disclose only their name, degree title, and university name to the employer. Attributes such as their student ID and graduation year remain hidden. The employer receives only the chosen attributes alongside their salts for hash verification, while hidden attributes appear only as opaque SHA-256 hashes that reveal nothing about their values.

2.4 Cryptographic Foundations

The prototype relies on two fundamental cryptographic building blocks: RSA digital signatures and SHA-256 hashing.

RSA Digital Signatures

RSA is a widely used public key cryptographic algorithm first described by Rivest, Shamir, and Adleman in 1978 (Rivest et al., 1978). It operates on the mathematical difficulty of factoring the product of two large prime numbers. In RSA, a private key is used to sign data and a corresponding public key is used to verify the signature.

In this project RSA works as follows. When the university issues a degree credential, it uses its RSA private key to sign the credential. The private key is kept secret and only the university has access to it. When an employer wants to verify the credential, they use the university's RSA public key, which is openly available, to check the signature. If the signature is valid, the employer knows two things: the credential was genuinely issued by the university, and nothing in the credential has been changed since it was signed.

SHA-256 Hashing

SHA-256 is a cryptographic hash function that takes an input of any length and produces a fixed 256 bit output (Preneel, 2010). It has two important properties for this project. First, it is a one way function, meaning the original value cannot be recovered from the hash. Second, it is collision resistant, meaning it is computationally infeasible for two different inputs to produce the same hash output.

In this project SHA-256 is used to protect undisclosed credential attributes. When the university issues a credential, each claim is combined with a unique random salt and hashed using SHA-256. For example, the claim student ID TU20250001 is combined with a random salt and hashed into a fixed string that reveals nothing about the original value. When the graduate presents the credential to an employer and chooses to hide certain attributes, only the hash is sent for those attributes. The employer receives no information about the hidden values but can still verify the overall credential signature.

How They Work Together

Together RSA and SHA-256 create a system where the credential is both tamper proof and privacy preserving. The SHA-256 hashes protect individual claim values from being revealed when not needed. The RSA signature ensures that the collection of hashes has not been altered since the university signed them. If anyone changes even a single character in any claim, the hash changes, the signed string no longer matches, and verification fails.

2.5 Related Work

Several projects and research efforts have explored similar approaches to distributed credential verification and selective disclosure.

Flamini et al. (2024) provide a comprehensive analysis of cryptographic mechanisms for selective disclosure in verifiable credentials. Their work compares atomic credentials, hiding commitment schemes, and SD signatures across multiple dimensions including privacy, complexity, and suitability for real world deployment. Their findings directly informed the design choices made in this project, particularly the selection of the commitment list mechanism as the most appropriate approach for a prototype level implementation.

The EU Digital Identity Wallet initiative represents the largest real world deployment of these principles. The European Commission requires that the wallet support selective disclosure so that citizens can share only the minimum necessary personal data in each transaction (European Commission, n.d.). This directly aligns with the privacy goal of this prototype and demonstrates that the technical approach explored here is relevant to current policy and real world systems.

Mazzocca et al. (2024) provide a broad survey of decentralized identifiers and verifiable credentials, covering the technical standards, real world implementations, and open research challenges in the field. Their work highlights the growing importance of user controlled identity systems and the limitations of centralized approaches, providing academic grounding for the motivation of this project.

3. System Design

This section describes the design of the prototype, including the overall system architecture, the roles of each service, the structure of the degree credential, the selective disclosure mechanism, and the key design choices made during development.

3.1 System Architecture

The prototype is structured as three independent Django web services, each representing one role in the issuer-holder-verifier model. The services communicate with each other over HTTP and run on separate ports to simulate a distributed environment. Although all three services run locally in the prototype, their separation into independent processes reflects the structure of a real-world distributed system where each party would operate on separate infrastructure.

The three services are:

- Issuer Service (Port 8001) — represents the university. Responsible for generating and signing degree credentials.
- Holder Service (Port 8002) — represents the graduate. Stores the credentials in a digital wallet and creates selective presentations.
- Verifier Service (Port 8003) — represents the employer. Verifies the authenticity of presented credentials using the issuer's public key.

The flow between services follows three stages. In the issuance stage, the university signs and sends a credential to the graduate's wallet. In the presentation stage, the graduate manually selects which attributes to disclose and sends a verifiable presentation to the employer. In the verification stage, the employer fetches the university's public key and verifies the presentation. Figure 1 below illustrates this architecture.

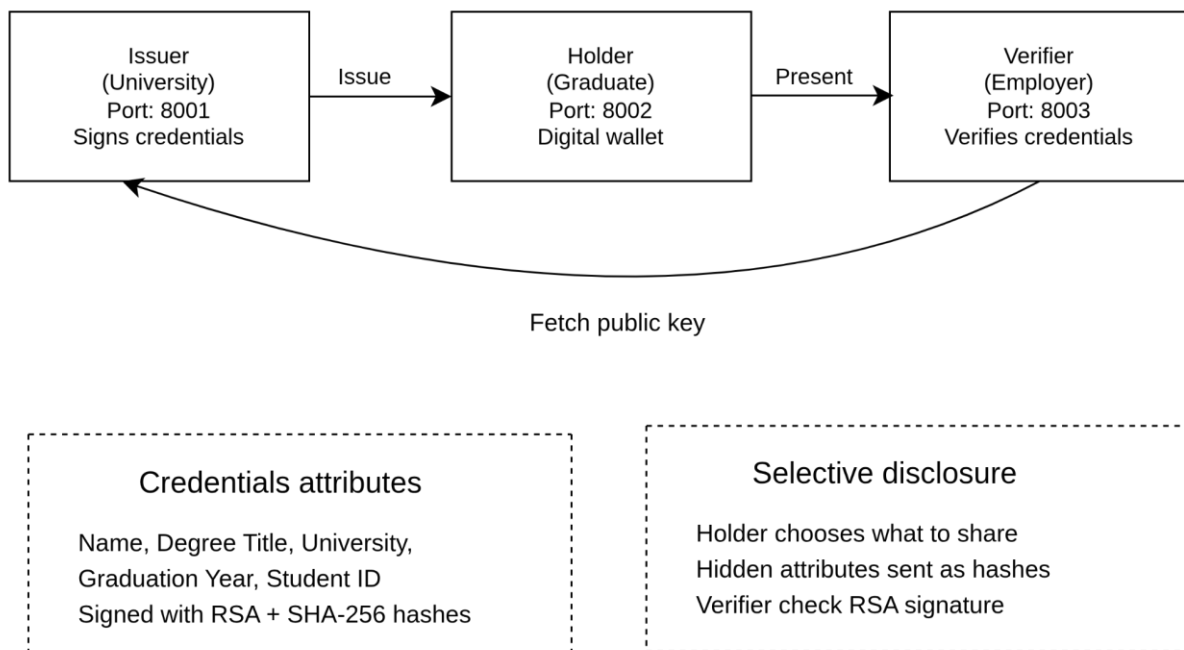


Fig 1, System architecture overview

3.2 The Roles

3.2.1 Issuer — University

The issuer is the trusted authority that creates and signs the degree credential. In this project, the issuer represents a university. When a student graduates, the university generates a digital credential containing all relevant claims, assigns a unique random salt to each claim, computes the SHA-256 hash of each salted claim, sorts all hashes alphabetically by claim key, and signs the resulting string using its RSA private key. The signed credential along with all claim values and salts is then sent directly to the holder's wallet.

The issuer also exposes a public key endpoint at `/issuer/public-key/` from which the verifier can retrieve the RSA public key for signature verification. This endpoint acts as a simplified verifiable data registry, consistent with the W3C Verifiable Credentials model (Sporny et al., 2025).

3.2.2 Holder — Graduate

The holder is the graduate who owns the credential. The holder's digital wallet stores the credential securely and displays all attributes to the graduate. When the graduate wishes to present the credential to an employer, they manually select which attributes to disclose from the wallet. The wallet then constructs a verifiable presentation containing the disclosed attributes with their salts, the hashes of all undisclosed attributes, and the original issuer signature. This presentation is sent to the verifier.

The holder has full control over what is disclosed in each transaction. This directly addresses the over-disclosure problem in the current TU verification process, where the graduate has no say over what personal data is shared.

3.2.3 Verifier — Employer

The verifier is the employer who needs to confirm the graduate's qualification. The verifier fetches the university's RSA public key from the issuer service, recomputes the SHA-256 hashes for all disclosed attributes, uses the provided hashes directly for undisclosed attributes, sorts all hashes alphabetically by hash value in the same alphabetical order as the issuer, and verifies the RSA signature against the reconstructed hash string. If verification succeeds, the employer can trust that the disclosed attributes are genuine and have not been tampered with since the university signed them.

3.3 Credential Structure

A degree credential in this system contains five attributes. Table 1 shows each attribute, an example value, and whether the holder can choose to hide it during presentation.

Attribute	Example Value	Can be hidden?
name	James Bond	No — always disclosed
degree_title	Bachelor of Computer Science	No — always disclosed
university	Tribhuvan University	No — always disclosed
graduation_year	2024	Yes
student_id	TU20250001	Yes

Table 1: Credential attribute

The attributes name, degree_title, and university are always disclosed because they represent the minimum information required for a meaningful degree verification. The remaining attributes, graduation_year and student_id, can be hidden by the holder depending on what the employer actually needs.

When the credential is issued, each attribute is individually salted and hashed. The issuer signs all hashes together rather than the raw values. This means the issuer's signature covers a commitment to all claims, but individual values can be withheld from the verifier without breaking the signature, as long as the verifier still receives the hash of the withheld claim.

3.4 Selective Disclosure Mechanism

The selective disclosure mechanism used in this project is the hiding commitment approach, specifically a commitment list scheme. The process works in three stages as follows.

During Issuance

The university generates a unique random salt for each claim. It computes SHA-256(claim value + salt) for each attribute. All resulting hashes are collected into a list and sorted alphabetically by the hash value. The sorted hashes are joined into a single string using a separator and signed using the issuer's RSA private key. The full credential, including all raw claim values, salts, hashes, and the signature, is sent to the holder's wallet.

During Presentation

The holder manually selects which attributes to disclose. For each disclosed attribute, the presentation includes the raw value and salt. For each hidden attribute, the presentation includes only the previously computed hash. The original issuer signature is always included in every presentation regardless of what the holder chooses to disclose.

During Verification

The verifier recomputes SHA-256(value + salt) for each disclosed attribute and confirms it matches the provided hash. For hidden attributes, the verifier uses the provided hash directly without any recomputation. The verifier then sorts all hashes alphabetically in the same deterministic order the issuer used, reconstructs the signing string, and verifies the RSA signature using the issuer's public key. If the signature is valid and all recomputed hashes match, the employer can confirm the credential is genuine and untampered.

3.5 Design Choices

This section explains the key technical decisions made during the design of the prototype and the reasoning behind each choice.

Choice of Selective Disclosure Mechanism

The commitment list mechanism was chosen for this prototype because it demonstrates selective disclosure using standard cryptographic tools, specifically RSA and SHA-256, that are already widely available in existing libraries. Unlike atomic credentials, the commitment list approach keeps all claims within a single signed credential, maintaining a cryptographic link between them. Unlike SD-Signatures which require zero knowledge proofs, the commitment list approach is straightforward to implement without advanced cryptographic constructs. This choice aligns with the project goal of demonstrating that selective disclosure is achievable using currently available technologies (Flamini et al., 2024).

Choice of Django Framework

Django was chosen as the web framework for all three services because it enables rapid development of independent HTTP services in Python. Each service is implemented as a separate Django project running on its own port, simulating a distributed microservice environment. Django's built-in HTTP request handling makes it straightforward to implement the API endpoints needed for credential issuance, wallet management, and verification.

Choice of RSA over Advanced Signature Schemes

RSA was chosen because it is a widely understood, well-documented, and readily available cryptographic standard. More advanced signature schemes such as BBS+ signatures would offer additional privacy properties such as unlinkability, but implementing them is significantly more complex and would require specialist cryptographic libraries. Since the primary goal of this prototype is to demonstrate selective disclosure rather than achieve full privacy compliance, RSA provides a sufficient and practical foundation.

Backend Only Implementation

The prototype includes minimal browser-based interfaces for the issuer dashboard and holder wallet, served directly by Django templates. These interfaces demonstrate the credential issuance and selective disclosure flow without requiring a separate frontend framework

4. Implementation

This section describes the implementation of the three services in the prototype: the issuer service, the holder wallet service, and the verifier service. Each service is implemented as an independent Django project. The section explains what each service does, which endpoints it exposes, and how the core logic works based on the

actual code. The section concludes with a description of the complete authentication flow.

Table 4 provides an overview of all HTTP endpoints exposed by the three services.

Service	Method	Endpoint	Purpose
Issuer	GET	/admin/dashboard/	Credential dashboard UI
Issuer	POST	/api/v1/issue/	Issue and sign a credential
Issuer	GET	/api/v1/pki/public-key/	Return RSA public key
Holder	GET	/wallet/ui/	Wallet interface UI
Holder	GET	/api/v1/wallet/credentials/	List stored credentials
Holder	POST	/api/v1/wallet/receive/	Receive credential from issuer
Holder	POST	/api/v1/wallet/presentation/generate/	Generate selective presentation
Verifier	GET	/api/v1/verify/	Audit log UI
Verifier	POST	/api/v1/verify/	Verify a presentation
Verifier	GET	/api/v1/verify/audits	List verification audit events

Table 2: HTTP Endpoints

4.1 Issuer Service

The issuer service runs on port 8001 and represents the university. It is implemented in the issuer_core Django project and contains three main components: the crypto module, the views module, and the dashboard template.

4.1.1 Key Management

The issuer service manages RSA key pairs through the `get_or_create_active_keypair` function in `crypto.py`. When the service starts, it checks the database for an existing active key pair. If none exists, it generates a new 2048-bit RSA key pair using a public exponent of 65537, stores both the private and public keys as PEM strings in the database, and marks the key pair as active. This means the same key pair is reused across service restarts, ensuring that previously issued credentials remain verifiable.

The key generation uses the following parameters:

- Key size: 2048 bits
- Public exponent: 65537

- Private key format: PKCS8
- Public key format: SubjectPublicKeyInfo

The public key is exposed at the endpoint `/api/v1/pki/public-key/` so that the verifier can fetch it during credential verification. The response includes the issuer origin, key type, algorithm identifier PSS-MGF1-SHA256, and the public key in PEM format.

4.1.2 Credential Issuance

When the university administrator visits the dashboard at `/admin/dashboard/` and submits a set of claims, the `issue_credential` view processes the request through the following steps.

First, the `salt_and_hash_claims` function processes each claim. For each claim key and value, it generates a random 16-byte hexadecimal salt using `secrets.token_hex(16)`, computes `SHA-256(value + salt)` as a hexadecimal digest, and stores the result as a `HashedClaim` dataclass containing the original value, the salt, and the hash. This produces a protected claims dictionary where each attribute is represented by all three fields.

```
def salt_and_hash_claims(raw_claims: dict[str, object]) -> dict[
    str, dict[str, str]]:
    protected_claims: dict[str, dict[str, str]] = {}
    for key, value in raw_claims.items():
        normalized_value = str(value)
        salt = secrets.token_hex(16)
        protected_claims[str(key)] = HashedClaim(
            value=normalized_value,
            salt=salt,
            hash=compute_claim_hash(normalized_value, salt),
        ).__dict__
    return protected_claims
```

Figure 2: `salt_and_hash_claims`

Second, the `compile_hash_payload` function extracts all hash values from the protected claims, sorts them alphabetically, and joins them into a single byte string using a null byte as the separator. This deterministic sorting ensures the verifier can reconstruct the same payload independently.

```
def sign_payload(payload: bytes, private_key_pem: str) -> str:
    private_key = serialization.load_pem_private_key(
        private_key_pem.encode("utf-8"),
        password=None,
    )
    signature = private_key.sign(
        payload,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH,
        ),
        hashes.SHA256(),
    )
```

Figure 3: sign_payload

Third, the `sign_payload` function signs the compiled payload using RSA-PSS with MGF1-SHA256 and maximum salt length. The resulting signature is stored as a hexadecimal string.

Finally, the credential is assembled as a structured JSON object following the W3C Verifiable Credentials format, including context, unique identifier, type, issuer origin, issuance date, credential schema, credential subject containing all protected claims, and a proof block containing the RSA-PSS signature. The credential is returned with HTTP status 201 and simultaneously pushed to the holder wallet at `/api/v1/wallet/receive/`.

4.2 Holder Service

The holder service runs on port 8002 and represents the graduate's digital wallet. It is implemented in the `holder_wallet` Django project and contains three main components: the views module, the redaction module, and the wallet template.

4.2.1 Credential Storage

When a credential arrives from the issuer at `/api/v1/wallet/receive/`, the `receive_credential` view first calls `validate_credential_structure` to confirm the credential contains all required fields. The validation checks that the credential contains the required top-level keys including context, id, type, issuer, credentialSubject, and proof. It also checks that each claim block within `credentialSubject.claims` contains value, salt, and hash fields. If validation passes, the credential is stored in the `WalletCredential` database model along with the issuer origin and credential identifier.

The graduate can view all stored credentials through the wallet UI at `/wallet/ui/` or retrieve them programmatically through the `list_credentials` endpoint at `/api/v1/wallet/credentials/`. The wallet displays all credential attributes including their raw values, giving the graduate full visibility of what is stored.

4.2.2 Selective Presentation

When the graduate selects which attributes to disclose and submits through the wallet UI, the `generate_presentation` view calls the `build_selective_presentation` function from `redaction.py` with the wallet credential identifier and a list of disclosed keys.

The `build_selective_presentation` function works as follows. It first validates the credential structure. It then creates a deep copy of the credential to avoid modifying the stored original. For each claim in the credential, it checks whether the claim key is in the disclosed keys list. If the claim is not disclosed, it removes the value and salt fields from that claim block, leaving only the hash. If the claim is disclosed, all three fields — value, salt, and hash — are retained.

```
def build_selective_presentation(credential: dict, disclosed_keys:
list[str]) -> dict:
    validate_credential_structure(credential)
    disclosed = {str(key) for key in disclosed_keys}
    reduced_credential = deepcopy(credential)
    claims = reduced_credential["credentialSubject"]["claims"]

    for claim_name, claim_block in claims.items():
        if claim_name not in disclosed:
            claim_block.pop("value", None)
            claim_block.pop("salt", None)

    return {
        "@context": ["https://example.local/ssi/presentation/v1"],
        "id": f"urn:uuid:{uuid4()}",
        "type": ["VerifiablePresentation",
            "SelectiveDisclosurePresentation"],
        "holder": "http://127.0.0.1:8002",
        "created": datetime.now(timezone.utc).isoformat(),
        "verifiableCredential": reduced_credential,
    }
```

Figure 4: `build_selective_presentation`

The function then wraps the reduced credential in a verifiable presentation object containing a unique presentation identifier, the holder origin, a creation timestamp, and the modified credential as `verifiableCredential`. The original RSA-PSS signature from the issuer proof block is always included regardless of which claims are disclosed, because the verifier needs it to check the credential's authenticity.

This mechanism directly implements the commitment list selective disclosure approach. Hidden attributes are represented only by their hashes, which reveal nothing about the original values because SHA-256 is a one-way function. The verifier can still check all hashes against the signature because the hash of each hidden claim is preserved in the presentation.

4.3 Verifier Service

The verifier service runs on port 8003 and represents the employer. It is implemented in the `verifier_node` Django project and contains two main components: the views module and the utils module. The service also maintains a `VerificationAudit` log that records the result of every verification attempt.

4.3.1 Verification Logic

When a verifiable presentation arrives at `/api/v1/verify/`, the `verify_presentation` view passes it to the `safe_verify` function in `utils.py`, which wraps the full verification logic in a try-except block to handle all possible failure cases gracefully.

The core verification is performed by `verify_presentation_document`, which follows these steps.

First, the function extracts the `verifiableCredential` from the presentation and retrieves the issuer field. It validates that the issuer points to the trusted local issuer at 127.0.0.1 port 8001. This prevents the verifier from being tricked into fetching a public key from an untrusted source.

Second, the `evaluate_claims` function processes each claim in the credential subject. For each claim, it checks whether value and salt fields are present. If both are present, the claim is disclosed and the verifier recomputes `SHA-256(value + salt)` and confirms it matches the provided hash. If neither is present, the claim is hidden and the provided hash is used directly. If only one of value or salt is present, this indicates partial disclosure metadata and the verification fails immediately. All evaluated hashes are collected into a list and disclosed claim values are recorded separately.

```
def evaluate_claims(claims: dict) -> tuple[list[str], dict[str, str]]:
    evaluated_hashes: list[str] = []
    disclosed_claims: dict[str, str] = {}

    if not isinstance(claims, dict) or not claims:
        raise VerificationError("Presentation contains no credential claims.")

    for claim_name, claim_block in claims.items():
        if not isinstance(claim_block, dict):
            raise VerificationError(f"Claim '{claim_name}' must be an object.")

        has_value = "value" in claim_block
        has_salt = "salt" in claim_block
        claim_hash = claim_block.get("hash")
        if not isinstance(claim_hash, str) or not claim_hash:
            raise VerificationError(f"Claim '{claim_name}' is missing a hash.")
```

Figure 5: evaluate_claims

Third, the `compile_hash_payload` function sorts the evaluated hashes alphabetically and joins them with a null byte separator, reconstructing the exact same payload the issuer signed.

Fourth, the verifier fetches the issuer's RSA public key by calling the `/api/v1/pki/public-key/` endpoint on the issuer service. The `fetch_issuer_public_key` function validates that the returned payload contains a valid PEM public key string.

Fifth, the `verify_signature` function converts the hexadecimal signature from the credential proof back to bytes and calls the RSA-PSS verify method using PSS padding with MGF1-SHA256 and maximum salt length. If the signature is valid, the function returns without raising an exception. If the signature is invalid, an `InvalidSignature` exception is raised and caught by `safe_verify`, which returns a failed verification result with a clear reason message.

4.3.2 Audit Logging

After every verification attempt, the verifier saves an audit record to the `VerificationAudit` database model. The record contains the presentation identifier, credential identifier, issuer origin, verification result, reason for failure if applicable, and the disclosed claims. The audit log can be viewed through the audit UI at `/api/v1/verify/` via GET request, or retrieved programmatically at `/api/v1/audit/`. The audit log stores the last 25 verification events and provides full transparency over all verification activity.

4.4 Authentication Flow

This section describes the complete end-to-end authentication flow of the prototype, from credential issuance by the university to verification by the employer. The flow demonstrates how the three services interact with each other and how selective disclosure is applied at each stage. Figure 2 illustrates this flow visually.

Stage 1 — Credential Issuance

The flow begins at the issuer service. The university administrator visits the dashboard at <http://127.0.0.1:8001/admin/dashboard/> and enters the graduate's credential claims, including name, degree title, university, graduation year, and student ID. When the administrator selects Sign and Export, the issuer service generates a unique random salt for each claim, computes SHA-256(value + salt) for each attribute, sorts all hashes alphabetically by claim key, and signs the resulting sorted payload using RSA-PSS with SHA-256. The complete credential, including all claim values, salts, hashes, and the RSA-PSS signature, is then automatically pushed to the holder wallet service at <http://127.0.0.1:8002/api/v1/wallet/receive/>.

Stage 2 — Selective Presentation

Once the credential is stored in the wallet, the graduate visits the wallet interface at <http://127.0.0.1:8002/wallet/ui/> where all credential attributes are displayed. The graduate manually selects which attributes to disclose to the employer. When the graduate selects Compile Presentation and Submit, the holder service constructs a verifiable presentation. For each disclosed attribute, the presentation includes the raw value and its salt. For each hidden attribute, the presentation includes only the previously computed hash. The original RSA-PSS signature from the issuer is always included. The completed verifiable presentation is then submitted to the verifier service at <http://127.0.0.1:8003/api/v1/verify/>.

Stage 3 — Verification

Upon receiving the verifiable presentation, the verifier service fetches the issuer's RSA public key from <http://127.0.0.1:8001/api/v1/pki/public-key/>. For each disclosed attribute, the verifier recomputes SHA-256(value + salt) and confirms it matches the provided hash. For each hidden attribute, the provided hash is preserved directly without recomputation. The verifier then rebuilds the sorted payload in the same deterministic alphabetical order the issuer used, and verifies the RSA-PSS signature using the issuer's public key. If the signature is valid and all recomputed hashes match, the credential is confirmed as authentic and the employer is granted access to the disclosed attributes. If the signature is invalid or any hash does not match, verification fails and the credential is rejected.

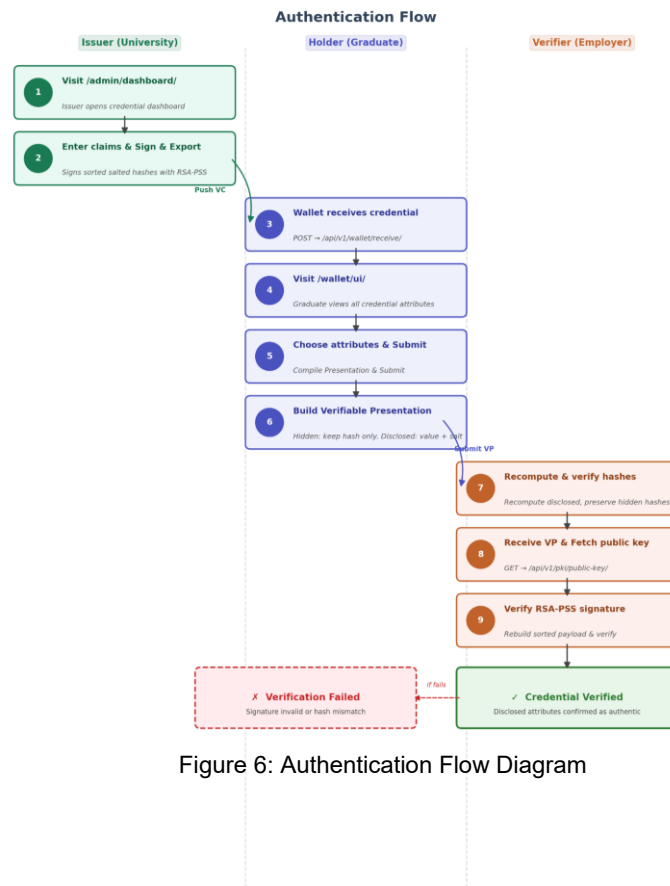


Figure 6: Authentication Flow Diagram

The complete flow demonstrates that a graduate can present only the attributes required for a specific transaction while the employer can still verify the authenticity of the entire credential through the issuer's RSA-PSS signature. The university is involved only once during issuance and is not contacted during any subsequent verification, giving the graduate full control over their personal data in each interaction.

5. Evaluation

This section evaluates the prototype against the privacy and technical goals identified in the introduction and literature review. Since this is a prototype level implementation, the evaluation is based on a design level analysis of the system rather than formal empirical testing. Each subsection examines what the system achieves, what it does not achieve, and why. The section concludes with a comparison between the proposed decentralized approach and the existing centralized degree verification process at Tribhuvan University.

Table 2 provides a summary of the evaluation results before the detailed discussion.

Requirement	Status	Method
Selective Disclosure	Achieved	Commitment list with SHA-256 and RSA
Data Integrity	Achieved	RSA digital signature
Data Minimisation	Achieved	Holder controls disclosed attributes
Unlinkability	Not achieved	Limitation of salt and hash method
Non-transferability	Not achieved	Outside scope of prototype

Table 3: Evaluation Summary

5.1 Selective Disclosure

Selective disclosure is the primary privacy goal of this project. The system is designed so that the credential holder, the graduate, can choose which attributes to reveal in each verification interaction. The verifier receives only those chosen attributes, while the remaining attributes are represented only as SHA-256 hashes that reveal nothing about their underlying values.

In the context of TU degree verification, a graduate applying for a job can choose to disclose only their name, degree title, and university name to an employer. Attributes such as graduation year and student ID remain hidden. The employer receives the disclosed attributes alongside their salts for hash recomputation, and receives only the hashes of the hidden attributes. The employer can verify the authenticity of the disclosed attributes without learning anything about the hidden ones.

From a design perspective, the commitment list mechanism ensures this property holds correctly. The issuer signs all claim hashes together, meaning the signature covers a commitment to all attributes. When the holder hides an attribute by withholding its value and salt, the verifier cannot recover that value because SHA-256 is a one-way function. The verifier can still check the signature because the hash of the hidden attribute is included in the presentation.

This directly addresses the over-disclosure problem described in the problem statement. Unlike the current TU process where the full academic transcript is submitted, this system gives the graduate genuine control over what personal data is shared in each transaction. This satisfies the GDPR principle of data minimisation, which requires that only the minimum necessary personal data be shared for a given purpose.

5.2 Data Integrity and Unforgeability

Data integrity means that a credential cannot be tampered with or modified after it has been issued. The system addresses this through RSA digital signatures. When the university issues a credential, it signs a deterministically sorted string of all claim hashes using its RSA private key. Any modification to the credential after issuance will result in a different hash, causing signature verification to fail.

Specifically, if a graduate attempts to change their degree title from Bachelor of Arts to Bachelor of Computer Science after the credential has been issued, the SHA-256 hash of that claim will produce a different value. When the verifier reconstructs the hash string and attempts to verify the RSA signature, the verification will fail because the signed string no longer matches. The verifier will know the credential has been tampered with.

RSA signature forgery without the issuer's private key is computationally infeasible based on the mathematical difficulty of the RSA factoring problem (Rivest et al., 1978). This means a verifier can trust that a credential with a valid signature was genuinely issued by the university and has not been modified since issuance.

The RSA key pair is generated once and persisted in the issuer's SQLite database. It is reused across service restarts, meaning previously issued credentials remain verifiable. However, if the database file is deleted, a new key pair is generated, and all previously issued credentials become unverifiable."

5.3 Unlinkability

Unlinkability means that two different presentations of the same credential cannot be traced back to the same holder. This is an important privacy property in digital identity systems as it prevents verifiers from tracking or profiling a holder's activity across multiple transactions.

The prototype does not achieve full unlinkability. This is a known limitation of the salt and hash method. Because the RSA signature is fixed at the time of credential issuance and does not change between presentations, a pair of colluding verifiers could compare the signatures in two separate presentations and determine that they came from the same credential and therefore the same holder. Even if the graduate discloses different attributes to different employers, the identical signature acts as a persistent linking identifier.

This limitation is consistent with the findings of the European Telecommunications Standards Institute, which notes that the salt and hash approach provides only partial unlinkability (ETSI, 2024). Specifically, the verifier cannot link individual claim values

across presentations if different attributes are disclosed each time. However, the signature itself remains linkable.

Achieving full unlinkability would require more advanced cryptographic techniques such as re-randomizable signature schemes like BBS+ signatures, or zero knowledge proofs. These approaches are outside the scope of this prototype but represent an important direction for future development.

5.4 Non-Transferability

Non-transferability means that a credential holder cannot share their credential with another person for them to use as their own. In the current prototype, non-transferability is not implemented. Because the holder service has no key pair of its own, there is no cryptographic mechanism to bind the credential to a specific individual. Any party who obtains the credential could theoretically present it to a verifier.

This is acknowledged as a limitation of the current prototype. Implementing non-transferability would require holder binding, where the holder signs the presentation with their own private key, proving they are the legitimate holder of the credential. This would significantly increase the complexity of the system and was therefore considered outside the scope of this prototype, which focuses on demonstrating selective disclosure.

5.5 Centralized vs Decentralized Comparison

This subsection compares the proposed decentralized approach with the existing centralized degree verification process at Tribhuvan University to evaluate the extent to which the prototype gives the credential holder greater control over their personal data.

In the current TU process, the university maintains a centralized database of all graduate records. When an employer needs to verify a degree, they contact the university directly or request the graduate to submit a scanned copy of their full academic transcript. The graduate has no control over what information is shared, the verification depends entirely on the university being available and cooperative, and the full transcript is disclosed even when only a small portion of it is needed.

In the proposed system, the university issues the credential directly to the graduate's digital wallet at graduation. All subsequent verification interactions occur directly between the graduate and the employer, without involving the university. The graduate decides what to disclose in each interaction. The employer can verify the credential's authenticity through the issuer's public key without needing to contact the university at all.

Aspect	TU Centralized System	Proposed Prototype
Who controls data sharing?	University	Graduate
What is disclosed?	Full transcript	Only chosen attributes
Is issuer involved in verification?	Yes always	No
Can holder hide attributes?	No	Yes
Over-disclosure risk?	High	Low
Single point of failure?	Yes	No

Table 4: Centralized vs Decentralized Comparison

The comparison shows that the proposed system gives the graduate significantly more control over their personal data compared to the existing TU process. The holder decides what to share, when to share it, and with whom, without depending on the university for each verification. This represents a meaningful improvement in privacy and user autonomy for the credential holder.

However the decentralized approach also introduces challenges that the centralized system handles automatically. Credential revocation is one example. In the TU centralized system, if a degree is revoked, the university can update its database immediately and any subsequent verification will reflect that change. In the proposed decentralized system, once a credential is issued to the holder's wallet, there is no mechanism to revoke it. This is acknowledged as an important limitation and represents a key area for future development.

6. Conclusion

This project set out to design and implement a distributed digital credential verification prototype that demonstrates selective disclosure in a university degree verification scenario. The motivation was a real and practical privacy problem: at institutions such as Tribhuvan University in Nepal, graduates must submit their full academic transcript when applying for employment or further education, disclosing personal information that is entirely irrelevant to the verification request. This practice of over-disclosure represents a fundamental privacy limitation of centralized credential systems.

The prototype addresses this problem by implementing the issuer-holder-verifier model defined by the W3C Verifiable Credentials specification (Sporny et al., 2025). Three independent Django web services represent the university, the graduate, and the employer. The university signs degree credentials using RSA digital signatures and salted SHA-256 hashes. The graduate stores the credential in a digital wallet and manually selects which attributes to disclose in each verification interaction. The employer verifies the authenticity of the disclosed attributes using the university's public key, without needing to contact the university directly.

The design level evaluation demonstrates that the prototype successfully achieves its primary goals. Selective disclosure is implemented through the commitment list mechanism, giving the credential holder genuine control over what personal data is shared in each transaction. Data integrity is ensured through RSA digital signatures, making the credential tamper-proof after issuance. Data minimisation is supported as a core design principle, with the holder able to disclose only the attributes required for a specific verification.

At the same time, the evaluation identifies clear limitations. Full unlinkability is not achievable with the salt and hash approach, as the fixed RSA signature can act as a linking identifier across presentations. Non-transferability is not implemented because the holder has no key pair of its own to bind the credential to their identity. Credential revocation is also outside the scope of the prototype. These limitations are consistent with the broader findings in the digital identity research literature and highlight the areas where more advanced cryptographic techniques would be needed for a production-grade system.

Despite these limitations, the prototype demonstrates that selective disclosure is achievable using currently available and widely understood technologies. A graduate can prove they hold a specific degree without revealing their full academic record. An employer can verify the authenticity of the disclosed attributes without accessing any information beyond what the graduate has chosen to share. This represents a meaningful step toward privacy-respecting credential verification compared to the existing centralized process.

6.1 Future Work

This prototype provides a foundation for several directions of future development.

- **Credential revocation:** A mechanism for the issuer to revoke credentials after issuance would be essential for a production system. This could be implemented using a revocation list or a status endpoint that verifiers check during verification.
- **Holder binding and non-transferability:** Assigning a key pair to the holder and requiring them to sign presentations would bind the credential to the legitimate holder and prevent credential theft or transfer.
- **Unlinkability:** Adopting more advanced signature schemes such as BBS+ signatures or zero knowledge proofs would allow the holder to generate unlinkable presentations, preventing verifiers from tracking holder activity across transactions.
- **Frontend interface:** Adding a user interface to the holder wallet would make the system more accessible and demonstrate the user experience of selective disclosure more clearly.

- Persistent key storage: The issuer's RSA key pair should persist across service restarts in a production system to ensure previously issued credentials remain verifiable.
- Blockchain integration: Storing issuer public keys and credential schemas on a blockchain would remove the need to trust the issuer's public key endpoint and provide a fully decentralized trust infrastructure.

7. References

Allen, C. (2016). The path to self-sovereign identity. Life With Alacrity Blog. Retrieved from <http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>

European Commission. (n.d.). EU Digital Identity. European Commission Digital Strategy. Retrieved from <https://digital-strategy.ec.europa.eu/en/policies/eudi-wallet>

European Telecommunications Standards Institute (ETSI). (2024). Electronic signatures and trust infrastructures; analysis of selective disclosure and zero-knowledge proofs applied to electronic attestation of attributes (ETSI TR 119 476 V1.2.1). https://www.etsi.org/deliver/etsi_tr/119400_119499/119476/01.02.01_60/tr_119476v010201p.pdf

Flamini, A., Sciarretta, G., Scuro, M., Sharif, A., Tomasi, A., & Ranise, S. (2024). On cryptographic mechanisms for the selective disclosure of verifiable credentials. ArXiv, abs/2401.08196. <https://doi.org/10.48550/arxiv.2401.08196>

Mazzocca, C., Acar, A., Uluagac, S., Montanari, R., Bellavista, P., & Conti, M. (2024). A survey on decentralized identifiers and verifiable credentials. IEEE Communications Surveys and Tutorials, 27, 3641-3671. <https://doi.org/10.1109/comst.2025.3543197>

Preneel, B. (2010). The first 30 years of cryptographic hash functions and the NIST SHA-3 competition. In Topics in Cryptology, CT-RSA 2010. Lecture Notes in Computer Science, 5985. Springer.

Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM, 21(2), 120-126.

Sporny, M., Longley, D., Chadwick, D., & Herman, I. (2025). Verifiable credentials data model v2.0. World Wide Web Consortium. <https://www.w3.org/TR/vc-data-model-2.0/>

Tribhuvan University. (n.d.). Official website. Retrieved from <https://www.tu.edu.np>

W3C. (2019). Verifiable credentials data model 1.0. World Wide Web Consortium. <https://www.w3.org/TR/vc-data-model/>